

The STACK & A REQUEST'S LIFE

tokio (runtime) → **hyper** (HTTP/1.1, HTTP/2) → **tower** (Service, Layer) → **axum** (Router, extractors, IntoResponse)

TcpListener → hyper connection → outer layers → Router (match path, method)

→ route layers → extractors (parts..., body last) → handler → IntoResponse → back out

CARGO.TOML & MAIN.RS

```
axum = "0.8.9"    tokio = { version = "1", features = ["full"] }
tower-http = { version = "0.7", features = ["trace", "timeout", "cors"] }
serde = { version = "1", features = ["derive"] } #
axum-extra = "0.12"

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let state = AppState::new().await?;
    let app = Router::new()
        .route("/health", get(|| async { "ok" })))
        .with_state(state);
    let listener =
    TcpListener::bind("0.0.0.0:3000").await?;
    axum::serve(listener, app)
        .with_graceful_shutdown(shutdown_signal())
    // ctrl_c + SIGTERM
        .await?;
    Ok(())
}
```

ROUTING

```
.route("/books", get(list).post(create))
.route("/books/{isbn}",
get(show).put(update).delete(remove))
.route("/files/{*path}", get(file)) //
wildcard
.route("/ws", any(ws)) //
WebSocket over h1 + h2
.nest("/api/v1", api_router) .merge(admin_router)
.nest_service("/assets", ServeDir::new("assets"))
.fallback(not_found)
.method_not_allowed_fallback(h405)
.route_service("/rpc", tower_service) // any
tower Service

Literal braces: {}. Nesting at / panics; merging two
routers with fallbacks panics (use reset_fallback).
```

EXTRACTORS

Type	From
Path<T> Query<T>	URL captures / query string (serde)
State<S> Extension<T>	router state / value set by middleware
HeaderMap TypedHeader<T>*	headers (*axum-extra)
Method Uri MatchedPath	request line, route template
ConnectInfo<SocketAddr>	needs into_make_service_with_connect_info
CookieJar* WebSocketUpgrade	cookies / upgrade
body: Json Form Bytes String Multipart Request	only one, and it must be the last argument

Option<T> needs OptionalFromRequestParts · Result<T, T::Rejection> inspects the rejection · #[debug_handler] explains trait errors.

RESPONSES

```
(StatusCode::CREATED, [(header::LOCATION, url)],
Json(book))
Html(markup) Redirect::see_other("/books")
NoContent
Sse::new(stream).keep_alive(KeepAlive::default())
/!([CONTENT_TYPE, "image/png"]), bytes) impl IntoResponse
Result<Json<Book>, AppError> // both sides
IntoResponse
```

APPERROR RECIPE

```
#[derive(Debug, thiserror::Error)]
enum AppError {
    #[error("not found")] NotFound,
    #[error(transparent)] Db{#[from] sqlx::Error},
}

impl IntoResponse for AppError {
    fn into_response(self) -> Response {
        let status = match &self {
            AppError::NotFound =>
                StatusCode::NOT_FOUND,
            AppError::Db(e) => { tracing::error!
(%e); StatusCode::INTERNAL_SERVER_ERROR }
        };
        (status, Json(json!({ "title":
status.canonical_reason() }))).into_response()
    }
} // handlers: Result<_, AppError> and `?`
everywhere; no unwrap()
```

STATE

```
#[derive(Clone, FromRef)] // FromRef:
"macros" feature
struct AppState { pool: PgPool, settings:
Arc<Settings> }
async fn h(State(pool): State<PgPool>) {} //
substate via FromRef
Router::new(...).with_state(state) // Router<S> →
Router<()>

Cheap to clone (pools, Arc). std::sync::Mutex unless
held across .await.
```

MIDDLEWARE

```
async fn auth(State(s): State<AppState>, mut req:
Request, next: Next)
-> Result<Response, StatusCode> {
    req.extensions_mut().insert(user);
    Ok(next.run(req).await) }
.layer(from_fn(timing))
.layer(from_fn_with_state(state.clone(), auth))
.route_layer(..) // only matched routes: 404 stays
404, not 401

Order: Router::layer — the last added runs first
(outermost); ServiceBuilder — top runs first. Layers
wrap routes added before them. Stateful layers
(ConcurrencyLimit, RateLimit, Buffer): serve
app.into_make_service().
```

TOWER-HTTP 0.7 & LIMITS

```
TraceLayer::new_for_http() SetRequestIdLayer +
PropagateRequestIdLayer
TimeoutLayer::with_status_code(StatusCode::REQUEST_T
IMEOUT, secs(10))
CompressionLayer RequestDecompressionLayer
CatchPanicLayer
SensitiveHeadersLayer
SetResponseHeaderLayer::if_not_present(...)
CsrfLayer (0.7) RequestBodyTimeoutLayer
Response/RequestBodyDeadlineLayer
ServeDir::new("dist").fallback(ServeFile::new("dist/
index.html"))
DefaultBodyLimit::max(10 * 1024 * 1024) // default
2 MB; ::disable()
CorsLayer::new().allow_origin(origin).allow_methods(
[GET, POST])

.allow_headers([CONTENT_TYPE]).allow_credentials(tru
e) // Any+credentials panics

Deadline layers go outside TimeoutLayer. axum::serve
0.8 has no header-read timeout: use a proxy or hyper-
util.
```

VALIDATION & OPENAPI

```
#[derive(Deserialize, Validate, ToSchema)]
struct NewBook { #[validate(length(min = 1, max =
200))] title: String
payload.validate().map_err(AppError::Validation)?;
// → 422
#[utoipa::path(get, path = "/books/{isbn}",
responses((status = 200, body = Book)))]
#[derive(OpenApi)] #[openapi(paths(show))] struct
ApiDoc;
SwaggerUi::new("/docs").url("/openapi.json",
ApiDoc::openapi())
```

SQLX 0.9

```
let pool = PgPoolOptions::new().max_connections(10)
.acquire_timeout(secs(3)).connect(&url).await?;
sqlx::migrate!().run(&pool).await?; //
./migrations
sqlx::query_as!(Book, "SELECT isbn, title FROM books
WHERE isbn = $1", isbn)
    .fetch_optional(&pool).await? // one
| all | optional
let mut tx = pool.begin().await?; q.execute(&mut
*tx).await?; tx.commit().await?;
cargo sqlx prepare → .sqlx/ + SQLX_OFFLINE=true
for CI and Docker
format!(...) SQL needs AssertSqlSafe (SqlSafeStr) · #
[sqlx::test] per-test DB
```

SECURITY CHECKLIST

☐ **Sessions:** tower-sessions, session.cycle_id() on login; cookies HttpOnly; Secure; SameSite=Lax, __Host-prefix

☐ **Passwords:** argon2 (Argon2id, PHC string) inside spawn_blocking; never store or return plain text

☐ **JWT:** jsonwebtoken 11 + aws_lc_rs/rust_crypto feature; check exp iss aud, fixed algorithm, require Bearer ; short-lived + refresh

☐ **OAuth2/OIDC:** authorization code + PKCE, state and nonce in a private cookie, validate the ID token

☐ **Authorization:** check ownership per resource (IDOR); 401 vs 403 vs 404

☐ **CSRF:** CsrfLayer or tokens for cookie auth · security headers (HSTS, CSP, nosniff)

☐ **TLS** at the proxy, or axum-server + rustls (install_default() the provider) · secrets from env, never in the image

TEMPLATES, HTMX, REAL TIME

```
#[derive(Template, WebTemplate)] #[template(path =
"book.html")]
struct BookPage { book: Book } //
askama + askama_web
if headers.contains_key("hx-request") { partial }
else { full page }
async fn ws(ws: WebSocketUpgrade) -> Response {
ws.on_upgrade(handle) }
while let Some(Ok(Message::Text(t))) =
socket.recv().await { ... } // Utf8Bytes
Sse::new(stream.map(|b|
Event::default().json_data(b))) // Result<Event, _>
while let Some(field) =
multipart.next_field().await? { field.file_name();
... }
```

OBSERVABILITY

```
tracing_subscriber::registry().with(EnvFilter::from_
default_env())
    .with(fmt::layer().json())

.with(tracing_opentelemetry::layer().with_tracer(pro
vider.tracer("web")))
    .init(); //
RUST_LOG=info,tower_http=debug
#[instrument(skip(state), fields(isbn = %isbn))]
async fn show(...)
SdTracerProvider::builder().with_batch_exporter(Spa
nExporter::builder()
    .with_tonic().build()?).build(); //
provider.shutdown() on exit
let h =
PrometheusBuilder::new().install_recorder()?; //
GET /metrics → h.render()
counter!("http_server_requests_total", "route" =>
route).increment(1);

Label by MatchedPath, never the raw path · W3C
traceparent in and out.
```

TESTING

```
let res =
app().oneshot(Request::get("/books/1").body(Body::em
pty()))?.await?;
assert_eq!(res.status(), StatusCode::OK);
let bytes =
res.into_body().collect().await?.to_bytes(); //
http-body-util
let server = TestServer::new(app());
// axum-test
server.get("/books/1").await.assert_status_ok().asse
rt_json_contains(&json!({..}));
#[sqlx::test] async fn t(pool: PgPool) {
    .layer(MockConnectInfo(addr))
```

DEPLOYMENT CHECKLIST

☐ cargo-chef multi-stage Dockerfile on a current rust:1.99-slim; runtime distroless/cc-debian13:nonroot

☐ no .env in the image; secrets from the platform · SQLX OFFLINE=true

☐ bind 0.0.0.0; /health (liveness) and /ready (readiness); metrics on an internal port

☐ SIGTERM → graceful shutdown < terminationGracePeriodSeconds; flush telemetry

☐ release: lto, codegen-units = 1, keep panic = "unwind" · blocking work in spawn_blocking

☐ Lambda: lambda_http::run(app) · Fly.io, Cloud Run, ECS: same image

Changed since older tutorials

axum::Server/hyper::Server → axum::serve(listener, app) (0.7)	/:id, /*rest → /{id}, /*{rest}; old syntax panics (0.8)	#[async_trait] on extractors → native async fn (0.8)	TypedHeader, Host → axum-extra (0.7, 0.8)
Option<T> extractor → OptionalFromRequestParts (0.8)	askama_axum → askama_web (askama 0.13+)	tower-http 0.7: TimeoutLayer::with_status_code, CsrfLayer	sqlx 0.9: SqlSafeStr, MSRV 1.94 · SeaORM 2 · jsonwebtoken 11